

第18章 虚机制

18.1 虚机制的引入

请看下面的代码：

```

1  #include <iostream>
2  class Shape {
3  public:
4      void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
5      double area( ) const { return 0; }
6  };
7  class Rectangle : public Shape {
8  public:
9      Rectangle( double w, double h ) { mWidth = w; mHeight = h; }
10     double area( ) const { return mWidth * mHeight; }
11 private:
12     double mWidth;
13     double mHeight;
14 };
15 class Circle : public Shape {
16 public:
17     Circle( double r ) : mRadius( r ) { }
18     double area( ) const { return mRadius * mRadius * 3.14; }
19 private:
20     double mRadius;
21 };
22 int main( ) {
23     Rectangle rect( 1, 2 );
24     Circle cir( 1 );
25     Shape & sp1 = rect;
26     Shape & sp2 = cir;
27     sp1.show( ); //输出 面积是 0, 为什么?
28     sp2.show( ); //输出 面积是 0, 为什么?
29 }
```

上述代码未能输出预期的面积值，但这并非由于程序编写错误，而是因为需要在语言层面上提供额外的语义支持。要正确输出面积值，程序在运行时必须能够识别 `sp1` 和 `sp2` 实际上是指向 `Rectangle` 和 `Circle` 对象的。然而，这种类型信息无法在编译期通过静态代码分析获得。例如，在函数 `void f(Shape & aShape)` 中，参数 `aShape` 的实际类型可以是 `Shape` 的任意子类，但无法在编译期仅从函数原型或实现中推断出 `aShape` 的具体类型。

为了解决这一问题，程序设计语言需要提供相应的解决机制。

18.2 静态编联和动态编联

静态编联，也称为早绑定或静态绑定，指的是在程序编译期间就确定了函数调用的具体版本。如果仅使用静态编联，那么程序中各个函数之间的调用关系是固定的，例如 C 语言中的函数调用就是静态编联。在上一节的代码中，由于使用了静态编联，在编译 `show( )` 中的 `this->area( )` 时，直接绑定为 `this->Shape::area( )`，因此运行结果为 0。

相比之下，动态编联，也称为晚绑定或动态绑定，则是在程序运行期间才决定具体调用哪个函数体。为了正确输出上例中的面积值，必须采用动态编联，即在运行时根据对象的实际类型确定 `this->area( )` 是调用 `Rectangle::area( )` 还是 `Circle::area( )`。

动态编联的实现方式有多种，对于像 C++ 这样需要静态编译的语言，最常见的实现方式是通过虚机制。C++ 中的虚机制包括虚函数、虚函数表、对象的内存布局、类型信息的存储和获取等多个方面。

18.3 虚函数

要使程序能够动态编联，可修改例子中 `Shape` 类的定义，如下：

```
1  class Shape {
2  public:
3      void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
4      virtual double area( ) const { return 0; } //改为虚函数
5      virtual ~Shape( ) = default;          //显式定义虚析构函数
6  };
```

修改后的程序将正确输出面积值。这里，给 `Shape::area( )` 添加了 `virtual` 关键字，使得所有子类中的 `area( )` 函数也都自动带 `virtual` 关键字；同时，自定义了 `Shape` 的析构函数且为 `virtual` 的，编译器不再提供 `Shape` 类缺省析构函数，而且 `Shape` 所有子类的析构函数也都自动变为 `virtual` 的。

18.3.1 虚函数

虚函数也称虚拟函数，虚函数的原型为：

格式： `virtual` 返回类型 函数名(参数列表) [`const`] [`异常规范`] [`override`];

实现虚函数时可以选择内联或外联形式，但如果不考虑程序优化，由于虚函数的动态绑定特性，

访问虚函数总是会被编译为函数调用。

与普通实例方法相比，虚函数除了需要添加 `virtual` 关键字外，在其他方面基本相同。需要注意的是，**虚函数必须是类的实例方法，而不能是静态类方法**；构造函数和拷贝构造函数的行为类似于静态方法，因此也不能声明为虚函数；**如果类中含有其他虚函数，则析构函数也应定义为虚函数**，以确保正确释放派生类的对象；赋值运算符通常不定义为虚函数；虚函数可以带有 `const` 修饰符，也可以不带；其访问控制可以根据需求任意设置。

18.3.2 派生类中的虚函数

虚函数总是与继承联系在一起，派生类中也可以有虚函数。由于保护继承和私有继承通常可用组合取代，因此在大多数语言中只支持公有继承，这里只讨论公有继承下的虚函数，也是虚函数的主要应用形式。实际上，对于其它继承方式，下面讨论的一样适用。

派生类中的虚函数主要为以下几种形式存在：

1. 在派生类中新定义的虚函数；
2. 继承自父类的虚函数，不包括虚析构函数；
3. 如果父类的析构函数是虚的，那么派生类中的析构函数也一定是虚的。此时派生类的析构函数，无论是编译器提供的缺省析构函数，还是自定义的析构函数，即使没有显式的 `virtual` 关键字，也是虚的。
4. 派生类中重写(override)的虚函数。此时的虚函数应与父类的虚函数同名，参数列表相同，返回类型相同或相容；这里的相容，也称协变，是指派生类的返回类型可以安全地向父类中的返回类型转换。
5. 派生类中与父类函数同名的虚函数，又分为重写(override)和覆盖(overwrite)两种：
 - (1) 若派生类中的虚函数与父类的虚函数同名，参数列表相同，`const` 修饰相同，异常说明相同，返回类型相同或相容，那么此时称为重写，这里的相容，也称协变(covariant return type)，是指派生类的返回类型可以与父类的返回类型不同，但必须是引用型或指针型，且可以安全地向父类中的返回类型转换。
 - (2) 若派生类中的虚函数与父类的虚函数同名，但参数列表、`const` 修饰或异常说明不完全相同，那么派生类的同名虚函数将覆盖(也称隐藏)父类的同名函数。

例如：

```
1 class Parent {  
2 public:
```

```

3     virtual ~Parent ( );
4     virtual Parent * f( );
5     virtual void g( );
6     int normal( );
7 protected:
8     virtual void h( );
9 };
10 class Child: public Parent {
11 public:
12     virtual ~Child ( ) override { }
13     virtual Child * f( ) override;
14     virtual int k( );
15 protected:
16     virtual void h( ) override;
17 };

```

在子类 Child 中，包含多个虚函数：公有的析构函数、f 函数、g 函数和 k 函数，以及受保护的 h 函数。其中，g 函数是从父类继承而来的，公有的析构函数、f 函数和受保护的 h 函数是在子类中重写的，而 k 函数则是子类新增的。

虽然 override 关键字虽是可选的，但显式使用可以清楚地告知编译器：这个函数意在重写父类中的同名虚函数(虚析构函数情况特殊)。这能有效避免因疏忽等原因导致的意外覆盖父类虚函数的问题。

定义虚函数只是在语法上告诉编译器应采用动态编联，但要实现动态编联，还需要虚函数表的支持。

18.4 虚函数表

虚函数表也称虚拟函数表、虚拟表、虚表，它是一个指针数组，其中存放类中各虚函数的入口地址。由于对象中只存储数据成员，不存储各函数的入口地址，所以一个类的虚函数表由该类的所有对象共享。在运行时刻，当首次实例化对象时，若类中有虚函数，则加载类的虚拟表。例如：

```

1 class Parent {
2 public:
3     Parent( );
4     virtual ~Parent ( ) { }
5     void f( );
6     virtual void g( );
7 protected:
8     virtual void k( );

```

```
9  };
10 class Child: public Parent {
11 public:
12     Child( );
13     virtual ~Child ( ) override { }    //override 可选, 建议使用
14     void other( );
15     virtual void g( ) override;        //override 可选, 建议使用
16 protected:
17     virtual void h( ) override;        //override 可选, 建议使用
18 };
```

这里的 Parent 类中有 3 个虚函数：Parent::~~Parent(), Parent::g(), Parent::k(); Child 类有 4 个虚函数：Child::~~Child(), Child::g(), Parent::k(), Child::h()。那么相应地 Parent 类和 Child 类虚拟表的表项内容将如图 18-1 和图 18-2 所示。

0	&Parent::~~Parent
1	&Parent::g
2	&Parent::k

图 18-1 Parent 类虚拟表的各项内容及索引

0	&Child::~~Child
1	&Child::g
2	&Parent::k
3	&Child::h

图 18-2 Child 类虚拟表的各项内容及索引

这里需要强调，在派生类的虚拟表中，对应父类部分的索引一定与父类的虚拟表一致，表项的最后部分才是派生类新定义的虚函数地址。这里，Child 类的虚拟表前三项一定对应父类虚拟表，即使在定义 Child 类时先定义 h() 函数，也是如此。

当实例化一个存在虚函数的类时，对象中除了有数据成员之外，还增加了一个虚拟表的指针，该指针指向该类的虚拟表，同类的多个对象共享这个虚拟表。此时的对象布局，如图 18-3 所示。

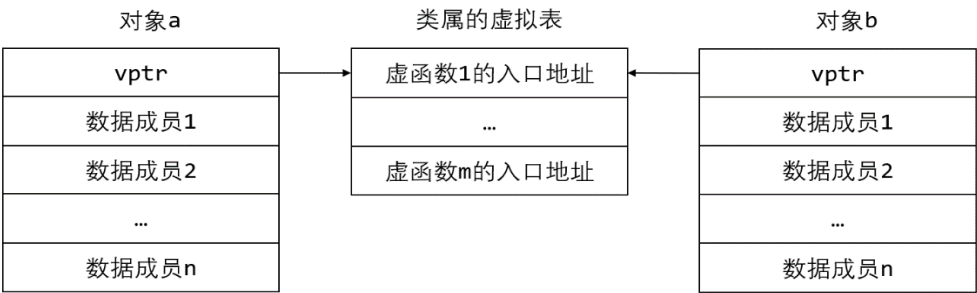


图 18-3 含虚拟表指针时对象的内存布局

vp_{tr}(Virtual Table Pointer)就是指向虚拟表的指针，对于不同的编译器，vp_{tr} 可放置于对象首位

置,也可能放于末尾,但无论哪种,编译器总是能够在编译时就确定其相对于首地址的偏移位置的。显然,一旦类中有虚函数,那么对象的内存布局将不同于没有虚函数的情况,当前各编译器普遍采用的就是这种入侵式的实现方式。

有了虚函数和虚拟表,再分析下边例子的执行过程:

```

1  #include <iostream>
2  class Shape {
3  public:
4      void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
5      virtual double area( ) const { return 0; }
6      virtual ~Shape( ) = default; //显式定义虚析构函数
7  };
8  class Rectangle : public Shape {
9  public:
10     Rectangle( double w, double h ) { mWidth = w; mHeight = h; }
11     virtual double area( ) const override { return mWidth * mHeight; }
12 private:
13     double mWidth;
14     double mHeight;
15 };
16 class Circle : public Shape {
17 public:
18     Circle( double r ) : mRadius( r ) { }
19     virtual double area( ) const override { return mRadius * mRadius * 3.14; }
20 private:
21     double mRadius;
22 };
23 int main( ) {
24     Rectangle rect(1,2);
25     Shape & rRect = rect;
26     Shape * pCircle = new Circle(3);
27     rRect.show( ); //正确输出 面积是 2
28     pCircle->show( ); //正确输出 面积是 28.26
29     delete pCircle;
30 }

```

具体执行过程如图 18-4 所示:

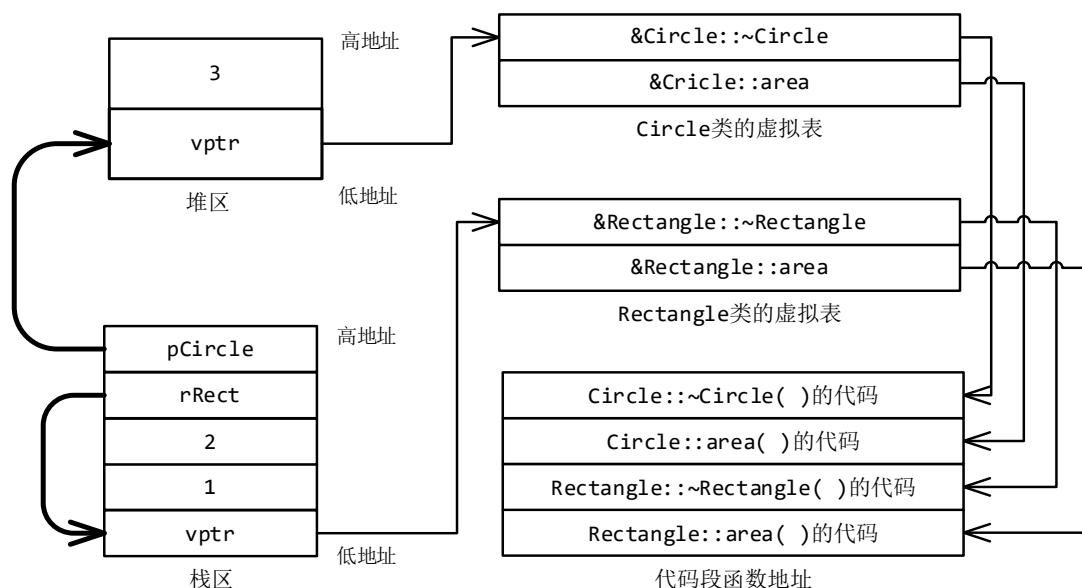


图 18-4 计算 shape 面积例子中各对象及执行过程示意图

执行 `main()` 时，先在栈区创建 `rect` 对象，其 `vptr` 指向 `Rectangle` 类的虚拟表；再创建引用变量 `rRect`，内部实现就是指向 `rect` 对象的指针；然后，在全局堆区创建 `Circle` 类的对象，其 `vptr` 指向 `Circle` 类的虚拟表，堆区对象的首地址存放到栈区变量 `pCircle`；执行 `sp1.show()` 时，会执行到 `area()` 函数，等价于 `this->area()`，通过 `this` 找到 `rect` 对象的 `vptr`，在虚拟表中找到 `area()` 函数的入口地址，即 `Rectangle::area()`，执行后返回 `Rectangle` 的面积；同理，执行 `sp2.show()` 时，也会正确输出 `Circle` 的面积；最后执行语句 `delete pCircle;`，需执行析构函数，由堆区中 `Circle` 对象的 `vptr`，得到析构函数入口地址：`&Circle::~~Circle`，执行后，由于 `Circle` 是子类，会自动执行 `Circle` 的父类 `Shape` 的析构函数。`main()` 函数退出时，释放栈区对象：先释放 `pCircle`，`pCircle` 就是一个指针，直接释放；再释放 `rRect`，内部实现就是指针，同样直接释放；再释放 `rect`，需要执行析构函数，由 `rect` 中的 `vptr` 得到 `&Rectangle::~~Rectangle`，执行后，由于 `Rectangle` 是子类，自动执行父类 `Shape` 的析构函数。最后结束程序。

上述的执行过程是手工分析的结果，要让程序自动执行上述的过程，需要编译器提供上述执行过程的代码，也就是让编译器生成能够实现动态编联的执行代码。下面针对动态编联的编译器实现做进一步讨论。

18.5 虚函数的动态编联

18.5.1 静态类型和动态类型

程序中的变量都是有类型的，每个变量的类型又分为静态类型和动态类型。

对于一个变量，在编译期间就可以确定的类型成为该变量的静态类型；而在运行时刻，才能确定的变量所对应的真实类型为动态类型。例如：

```
1  Child child;           //child 的静态类型为 Child, 动态类型也是 Child
2  Parent * pObj = &child; //pObj 的静态类型为 Parent * , 动态类型是 Child *
3  Parent & rObj = child;  //rObj 的静态类型为 Parent , 动态类型是 Child
4  Parent obj = child;     //obj 的静态类型为 Parent , 动态类型是 Parent
```

再例如：

```
1  int main( ) {
2      Rectangle rect(1,2); //rect 的静态类型是 Rectangle,动态类型也是 Rectangle
3      Shape & sh1 = rect;  //sh1 的静态类型是 Shape,动态类型是 Rectangle
4      Shape * psh2 = new Circle(3); //psh2 的静态类型是 Shape * ,动态类型是 Circle *
5      rect.show( );
6      sh1.show( );        //正确输出 2
7      psh2->show( );      //正确输出 28.26
8      delete psh2;
9      void userFunc( const Shape & );    //函数声明
10     userFunc( rect );    //实参 rect 的静态类型是 Rectangle,动态类型也是 Rectangle
11 }
12 void userFunc( const Shape & sh ) {
13     sh.show( ); //sh 的静态类型是 Shape,动态类型只能在运行时刻由实参确定
14 }
```

18.5.2 动态编联的编译实现

假设 ptr 是一个指向对象的指针，其静态类型为 Parent *，下面以访问 func 实例方法为例，说明语句 ptr->func() 的编译实现过程：

1. 如果 func 带有限定名，如 ptr->Parent::func()，那么直接采用静态编联，即编译时就确定调用 Parent 类中的 func 函数。
2. 如果 func 没有限定名，如 ptr->func()，则先确定 ptr 的静态类型，如 Parent *；
3. 在对应的静态类型中，如 Parent 类，寻找名字为 func，且函数原型及参数列表匹配的实例方法；
4. 如果没有找到，则编译报错，结束。
5. 如果找到，检查是否为虚函数；
6. 若为非虚函数，则直接采用静态编联，即等价于 ptr->Parent::func()；
7. 若为虚函数，则采用动态编联。即将语句 ptr->func()，编译成：

`(* ptr->vptr)[ index ] ( ( void * ) ptr , ... )`.其中, 对于 `vptr` 在对象内存布局中的偏移位置编译器是知道的; `index` 表示虚函数的入口地址在虚函数表中的索引号, 实际上这个 `index` 是 `func` 函数在 `Parent` 类型的虚拟表中的索引号, 由于派生类虚拟表中父类虚函数一定在前并且是一一对应的, 所以尽管虚函数 `func` 在父类和派生类中可能有不同的入口地址(也就是函数体), 但它们在父类和派生类中的虚拟表中的索引一定是一致的; `ptr` 的值就是实例方法中隐含的 `this` 指针值。因此`(* ptr->vptr)`表示取得虚函数表, `(* ptr->vptr)[index]`表示取得实际执行的虚函数地址, 实例方法的第一个参数是 `this` 指针, 值为 `ptr` 的值。这样, 通过编译后的代码可以让程序在运行时刻, 通过虚拟表取得真正执行的函数入口地址, 然后再执行了, 也就实现了动态编联或动态绑定。

从上边访问实例方法的编译过程可知: 若希望 `pObj->func()`; 或 `obj.func()`; 语句合法, 那么在 `pObj/obj` 对应的静态类型中必须存在匹配的实例方法 `func`, 即使静态类型中的 `func()` 函数永远都不会执行, 也要存在。例如, 前边 `Shape` 例子中 `Shape` 类中的 `virtual double area() const { return 0; }`, 在程序执行过程中, 不会执行该函数体, 但 `Shape` 类中必须有 `area()` 函数, 而且是虚的。

18.5.3 例1

```

1  class A {
2  public:
3      virtual ~A( ) { }
4      virtual void f2( ) { }
5      virtual void g( ) { }
6  };
7  class B: public A {
8  public:
9      virtual void f1( ) { }
10     virtual void f2( ) override { }
11     virtual void f3( ) { }
12 };
13
14 // 类 A 的虚拟表中有 3 项, 索引和入口地址为:
15 // 0  &A::~~A
16 // 1  &A::~f2
17 // 2  &A::~g
18
19 // 类 B 的虚拟表中有 5 项, 索引和入口地址为:
20 // 0  &B::~~B
21 // 1  &B::~f2
22 // 2  &A::~g

```

```

23 // 3  &B::f1
24 // 4  &B::f3
25
26 int main( ) {
27     A * p1 = new A; //p1 的静态类型为 A *
28     p1->f2( );      //编译成 (*p1->vptr)[1]((void *) p1 );
29     p1->g( );        //编译成 (*p1->vptr)[2]((void *) p1 );
30     //p1->f1( );     //编译错误! 静态类型类中无 f1 函数
31
32     A * p2 = new B; //p2 的静态类型为 A *
33     p2->f2( );      //编译成 (*p2->vptr)[1]((void *) p2 );
34     p2->g( );        //编译成 (*p2->vptr)[2]((void *) p2 );
35     //p2->f1( );     //编译错误! 静态类型类中无 f1 函数
36     delete p2;      //编译成 (*p2->vptr)[0]( (void *)p2 )
37     delete p1;      //编译成 (*p1->vptr)[0]( (void *)p1 )
38 }

```

18.5.4 例2

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Date {
5  public:
6      Date( int y, int m, int d ) :mYear( y ), mMonth( m ), mDay( d ) { }
7      virtual ~Date( ) { }
8      bool isLeap( ) const { /*其它略*/ return false; }
9      virtual string str( ) const { return ""; }
10 protected:
11     int mYear;
12     int mMonth;
13     int mDay;
14 };
15 class ChineseDate :public Date {
16 public:
17     ChineseDate( int y, int m, int d ) :Date( y, m, d ) { }
18     virtual string str( ) const override { return to_string( mYear ) + "年"
19         + to_string( mMonth ) + "月" + to_string( mDay ) + "日"; }
20 };
21 class AmericanDate : public Date {
22 public:
23     AmericanDate( int y, int m, int d ) :Date( y, m, d ) { }
24     virtual string str( ) const override
25     { return to_string( mMonth ) + "/" + to_string( mDay ) + "/" + to_string(mYear); }

```

```

26 };
27 class BritishDate :public Date {
28 public:
29     BritishDate( int y, int m, int d ) :Date( y, m, d ) { }
30     virtual string str( ) const override
31     {return  to_string( mDay ) + "/" + to_string( mMonth ) + "/" + to_string( mYear);}
32 };
33 class User {
34 public:
35     void showDate( const Date & date ) { cout << date.str( ) << endl; }
36 };
37 int main( ) {
38     ChineseDate d1( 2018, 12, 31 );
39     AmericanDate d2( 2018, 12, 31 );
40     BritishDate d3( 2018, 12, 31 );
41     User user;
42     user.showDate( d1 ); //输出 2018 年 12 月 31 日
43     user.showDate( d2 ); //输出 12/31/2018
44     user.showDate( d3 ); //输出 31/12/2018
45 }

```

18.6 虚函数的访问

在访问成员函数 `func` 时, 如果 `func` 函数是非虚函数或者成员函数使用了限定名, 如 `p->A::func()` 形式, 那么直接使用类 `A` 中的本地版本进行静态编联, 这里本地版本是指 `func` 函数是本类中存在的成员函数, 它可以是本类新定义的, 也可以是由祖先类继承而来的; 如果 `func` 是虚函数, 则 `func` 必定是实例方法, 若编译器不进行优化, 那么采用动态编联。

18.6.1 虚函数的本地版本

在构造函数和析构函数中访问虚函数时, 由于存在特殊原因, 不会采用动态编联, 而是会使用本地版本进行静态编联。考察如下代码:

```

1  class A {
2  public:
3      A( ):mA(100) { f( ); }
4      virtual ~A( ) { f( ); }
5      virtual void f( ){ cout<< mA; }
6  private:
7      int mA;
8  };
9  class B : public A {

```

```

10 public:
11     B( ):mB(200) { }
12     virtual ~B( ) { f( ); }
13     virtual void f( ) { cout << mB; }
14 private:
15     int mB;
16 };
17 int main( ) {
18     B b;
19 }

```

当实例化 B 类对象 b 时，会调用 B 的构造函数。此时，首先会调用基类的构造函数来构造基类子对象，并将 mA 初始化为 100。在基类构造完成后，程序会访问 f()。如果此时对 f() 的调用采用动态绑定，则会调用 B::f()，进而访问数据成员 mB。然而，在这时 B 类尚未完全构造完毕，mB 还未被初始化。可见，此时使用动态编联访问虚函数是不合适的。

同理，在析构对象 b 时，首先会析构 B 类的的数据成员，然后自动执行父类 A 的析构函数 A::~~A()，这时会访问 f()，若使用动态编联，将会执行 B::f()，会访问数据成员 mB，而此时 mB 已经释放了。可见，此时也不适合采用动态编联。

为了避免这种问题，C++ 标准规定，在构造函数或析构函数中调用函数时，无论是否声明为虚函数，都会采用本地版本进行静态编联。

下面的代码和运行结果，体现了虚函数和非虚函数的编译和执行过程。

```

1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      Base( ) { vf( ); }
6      virtual ~Base( ) { vf( ); }
7      virtual void vf( ) {
8          cout << "Base::vf( )" << endl;
9      }
10     virtual void vg( ) {
11         cout << "Base::vg( )" << endl;
12         vf( );
13         nvh( );
14     }
15     void nvh( ) {
16         cout << "Base::nvh( )" << endl;
17         vf( );
18     }

```

```

19 };
20 class Derived : public Base {
21 public:
22     Derived( ) { vf( ); }
23     virtual ~Derived( ) override { vf( ); }
24     virtual void vf( ) override { cout << "Derived::vf( )" << endl; }
25     void nvh( ) {
26         cout << "Derived::nvh" << endl;
27         vf( );
28     }
29     virtual void My( ) { }
30 };
31 int main( ) {
32     cout << "1.-----" << endl;
33     Base * p = new Derived;
34     cout << "2.-----" << endl;
35     p->vf( );
36     cout << "3.-----" << endl;
37     p->vg( );
38     cout << "4.-----" << endl;
39     p->nvh( );
40     cout << "5.-----" << endl;
41     delete p;
42 }

```

运行结果:

```

1.-----
Base::vf( )
Derived::vf( )
2.-----
Derived::vf( )
3.-----
Base::vg( )
Derived::vf( )
Base::nvh( )
Derived::vf( )
4.-----
Base::nvh( )
Derived::vf( )
5.-----
Derived::vf( )
Base::vf( )

```

18.6.2 私有的虚函数

在 C++ 语言中，虚函数的访问控制没有限制，甚至可以是私有的。但这种私有虚函数的执行结果可能有违访问控制的本意，请看如下代码：

```

1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      virtual ~A( ) { }
6      virtual void vPub( ) {
7          cout << "A::vPub( )" << endl;
8          vPriv( );
9          vProt( );
10     }
11 protected:
12     virtual void vProt( ) { cout << "A::vProt( )" << endl; }
13 private:
14     virtual void vPriv( ) { cout << "A::vPriv( )" << endl; }
15 };
16 class B : public A {
17 private:
18     virtual void vPriv( ) { cout << "B::vPriv( )" << endl; }
19 };
20 int main( ) {
21     A * pA = new B;
22     pA->vPub( ); //输出的三行分别为: A::vPub( ), B::vPriv( ) 和 A::vProt( )
23     delete pA;
24 }

```

语句 `pA->vPub( )` 经编译后，执行的是 `pA->A::vPub( )`，`A::vPub` 中又调用 `vPriv( )`，`vPriv` 在 `A` 中是虚的，采用动态编联，运行时，实际执行的是 `B::vPriv( )`。这就出现不合理的情况：`B::vPriv` 是私有，应该只能被本类的成员访问，在这里却由 `A` 类访问了。产生这种情况的根本原因在于，访问控制只能在编译时起作用，在运行时刻，由于虚拟表中的各表项只是虚函数的入口地址，不含有访问控制信息，所以访问控制在运行时刻是不起作用的。因此，对于私有的虚函数，虽然在 C++ 中仍是采用动态编联，但在其它面向对象语言中，却通常采用静态编联，即当编译 `p->vPriv( )` 时，若 `vPriv` 在 `p` 的静态类型中是私有的，那么无论是否为虚的，都将被编译成 `p->A::vPriv( )`。

18.6.3 final 修饰符

`final` 是一个关键字，用于修饰类或虚函数，表示它们不能被进一步继承或重写。

当用 `final` 修饰一个类时，将禁止从该类派生，这样的类也称 `final` 类。例如：

```
1 class Base final {
2     public:
3         void show( ) {
4             std::cout << "Base class" << std::endl;
5         }
6     };
7 // 以下代码会编译错误，因为 Base 是 final 类，不能被继承
8 class Derived : public Base {
9     };
```

`final` 类可以明确地禁止派生，从而避免被误用，如 `std::mutex` 就是 `final` 类。但 `final` 是一种强约束，过度使用 可能会限制代码的灵活性，使用时应确保类的设计确实不需要扩展。

还可以用 `final` 修饰类中的虚函数，这将禁止派生类中重写该虚函数。例如：

```
1 class Base {
2     public:
3         virtual ~Base( ) = default;
4         virtual void func( ) final { }
5         virtual void func(int) final { }
6         virtual void func(Base &) final { }
7         virtual void otherFunc( ) { }
8     };
9 class Derived : public Base {
10    public:
11        //void func(int) override { } //编译错误，func(int)是 final 函数，不能被重写
12        void otherFunc( ) override { } //正确
13    };
```

例子中 `Base::func` 是重载的，但由于都带有 `final` 修饰，在派生类中不能重写任何 `func`，也就避免了派生类覆盖父类 `func` 函数的可能性，而 `Base::otherFunc()` 没有 `final` 修饰，仍是可重写的。

`final` 类中是可以有虚函数的，但由于禁止派生，相当于 `final` 类中虚函数都带有 `final` 修饰。

18.7 抽象类

回顾 `Shape` 类的定义：

```
1 class Shape {
2     public:
3         virtual ~Shape( ) = default; //显式定义虚析构函数
4         void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
```

```

5     virtual double area( ) const { return 0; }
6 };

```

代码中实现了 `area()` 函数，但有几个特别的地方需要注意：首先，直接实例化 `Shape` 类对象是没有实际意义的，真正有意义的是其子类的对象。其次，给出的 `area()` 函数实现并未被调用，因此无论如何实现 `Shape::area()` 函数都不会影响最终的执行结果。最后，为了确保 `Shape::show()` 函数能够编译通过，`Shape` 类中必须包含一个虚函数 `area()`。基于上述原因，引入了纯虚函数。

18.7.1 纯虚函数

在定义虚函数时，如果在函数声明末尾添加 `=0`，则该虚函数被称为纯虚函数。纯虚函数一定是虚的实例方法，其访问控制可以是 `public`、`protected` 或 `private`。纯虚函数的主要作用是定义行为规范，强制派生类或后裔类在实例化时必须实现该接口。例如：

```

1  class Shape {
2  public:
3      virtual ~Shape( ) = default;
4      void show( ) const    { cout << "面积是:" << area( ) << endl; }
5      virtual float area( ) const = 0;    //纯虚函数
6  };

```

在类 `Shape` 中声明了一个公有的纯虚函数 `area()`，这表明 `Shape` 类不能被实例化。尽管其派生类具备实现 `area()` 行为的能力，但若实例化这些派生类，必须在派生类中具体实现 `area()` 函数。这就是抽象类的含义。

18.7.2 抽象类和具体类

如果一个类包含一个或多个纯虚函数，那么这个类就成为抽象类，抽象类是不能被实例化的。抽象类通常用于表示那些没有实例化意义的概念，例如“动物”、“水果”或“形状”等抽象名词；相对地，可以实例化的类被称为具体类。不过，并非所有无法实例化的类都是抽象类。例如，`class A { A() = delete; };` 中的类 `A` 虽然不能实例化，但它并不是抽象类。

定义抽象类的主要目的是为派生类提供高层次的类型抽象。尽管抽象类本身不能被实例化，但它为派生类设定了访问约定或接口规范，要求所有派生类都要遵守这些规范。特别是，若派生类实现了抽象基类中所有的纯虚函数，那么该派生类会成为具体类；若未实现全部纯虚函数或新增了纯虚函数，派生类仍为抽象类。同样地，具体类的派生类既可以是具体类，也可以是抽象类。例如，前面提到的 `Shape` 类和 `Date` 类都可以定义为抽象类：


```

1  //抽象类, 但不是接口类
2  class Shape {
3  public:
4      virtual ~Shape( ) = default;      //显式定义虚析构函数
5      void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
6      virtual double area( ) const = 0; //纯虚函数
7  };
8  //Rectangle 是具体类
9  class Rectangle : public Shape {
10 public:
11     Rectangle( double w, double h ) { mWidth = w; mHeight = h; }
12     virtual double area( ) const override { return mWidth * mHeight; }
13 private:
14     double mWidth;
15     double mHeight;
16 };
17 //Date 抽象类, 但不是接口类
18 class Date {
19 public:
20     Date( int y, int m, int d ) :mYear( y ), mMonth( m ), mDay( d ) { }
21     virtual ~Date( ) { }
22     bool isLeap( ) const { /*其它略*/ return false; }
23     virtual string str( ) const = 0; //纯虚函数
24 protected:
25     int mYear;
26     int mMonth;
27     int mDay;
28 };
29 //ChineseDate 是具体类
30 class ChineseDate :public Date {
31 public:
32     ChineseDate( int y, int m, int d ) :Date( y, m, d ) { }
33     virtual string str( ) const override
34     {return to_string( mYear ) + "年" + to_string( mMonth ) + "月" + to_string( mDay);
35     }
36 };

```

如果一个抽象类中, 除了静态成员、构造函数、拷贝构造函数、析构函数和赋值函数之外的所有成员函数均为纯虚函数, 则该抽象类被称为纯抽象类。其中, 析构函数必须是虚函数, 但不能是纯虚函数。例如:

```

1  //纯抽象类
2  class Fruit {

```

```

3 public:
4     static void staticMethod( ) { }
5 public:
6     Fruit( ) = default;          //可省略
7     Fruit( const Fruit & ) = default; //可省略
8     Fruit & operator=(const Fruit &) = default; //可省略
9     virtual ~Fruit( ) = default;    //虚析构，但不能是纯虚的
10 public:
11     virtual int growDays( ) const = 0;
12     virtual void plant( ) = 0;
13 };

```

18.7.3 接口类

如果一个抽象类是纯抽象类且没有实例变量，所有成员均为公有，则该抽象类也被称为**接口类**。例如，上述的 Fruit 类就是一个典型的接口类。从接口类的定义可知，接口类是一种特殊的抽象类，其目的在于为所有子类约定一套最小的公有访问接口。由于不涉及保护或私有成员，接口类能够为各子类提供更高层次的抽象。此外，由于接口类中没有实例变量，不会影响子类对象的内存布局，因此在多重继承中可以广泛使用接口类，既避免了数据成员命名冲突问题，也不影响子类向父类转换时的效率。

在定义接口类时，为便于理解和维护，习惯上将类名定义为 IXyz 的形式。例如：

```

1 //接口类
2 class IAreable {
3 public:
4     virtual ~IAreable( ) { }
5     virtual double area( ) const = 0;
6 };
7 //派生的接口类
8 class ICountry : public IAreable {
9 public:
10     virtual int population( ) const = 0;
11 };
12 //抽象类：存在纯虚函数 area( )
13 class Shape : public IAreable {
14 public:
15     void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
16 };
17 //仍是抽象类：area( )仍是纯虚函数
18 class Polygon : public Shape {
19 public:

```

```

20     Polygon( int s ) :mSides( s ) { }
21     int countSides( ) const { return mSides; }
22 private:
23     int mSides;
24 };
25 //Rectangle 是具体类
26 class Rectangle :public Polygon {
27 public:
28     Rectangle( double w, double h ) : Polygon( 4 ), mWidth( w ), mHeight( h ) { }
29     virtual double area( ) const override { return mWidth * mHeight; }
30 private:
31     double mWidth;
32     double mHeight;
33 };
34 //具体类
35 class Farm : public IAreable {
36 public:
37     virtual double area( ) const override { return 1000; }
38 };

```

18.7.4 纯虚定义

在抽象类中，纯虚函数也可以提供具体实现，这种实现被称为纯虚定义。例如：

```

1 //table.h 文件中
2 //抽象类-表格
3 class Table {
4 public:
5     virtual ~Table( ) = default;
6     virtual void show( ) const = 0; //纯虚函数也可以有实现
7     virtual float func( ) { /* 略 */ }
8 };
9 //具体类
10 class ExcelTable:public Table {
11 public:
12     virtual void show( ) const override {
13         Table::show( ); //调用 Table 中的纯虚定义
14         /* 其它略 */
15     }
16     virtual float func( ) override { /* 略 */}
17 };
18 //table.cpp 文件中
19 #include <iostream>
20 #include "table.h"

```

```
21 // 纯虚定义
22 void Table::show( ) const {
23     std::cout << "显示表格头部内容"<< std::endl;
24     std::cout << " =====" << std::endl;
25 }
```

含有纯虚函数的类必是抽象类，抽象类不能被实例化，因此无法通过对象直接访问纯虚函数，那么给出纯虚定义有何意义呢？实际上，纯虚定义的目的在于：首先，纯虚函数可以在抽象类中提供部分公共代码，允许派生类通过限定名调用这些代码，例如，上例中子类在实现时可通过 `Table::show( )` 调用父类的纯虚定义。其次，纯虚函数可以为子类的实现提供范本，使得在重写 (override) 该函数时，可以参考和借鉴抽象类中的纯虚定义，有助于以正确的逻辑和更规范的方式实现子类代码。

18.8 运行时类型识别

运行时类型识别(Run-Time Type Identification, RTTI)是一种在程序运行时确定对象类型的机制。它使程序能够在运行时检查和操作对象的类型信息，从而判断对象的实际类型，并安全地将其转换为特定类型。

要实现运行时类型识别，关键在于在运行期间获取对象的动态类型，这主要依赖于 `dynamic_cast` 运算符、`typeid` 运算符以及 `type_info` 结构。`dynamic_cast` 用于在继承层次结构中进行安全的类型转换；`typeid` 可以返回一个 `type_info` 对象，该对象包含了类型的相关信息；而 `type_info` 则记录了具体的类型信息，信息的具体格式可能因编译器的不同而有所差异。

当对对象应用 `typeid` 时，它会返回该对象的实际动态类型；同样地，`dynamic_cast` 需要利用动态类型信息来确定是否可以执行向下类型转换。因此，在使用运行时类型识别时，对象的动态类型信息必定被存储在某个地方。实际上，编译器通常将这些信息存储在虚拟表(vtable)中。除了虚函数的入口地址外，虚拟表还包含一个指向 `type_info` 对象的指针，该对象描述了该虚拟表所对应的类型的详细信息，如图 18-5 所示。

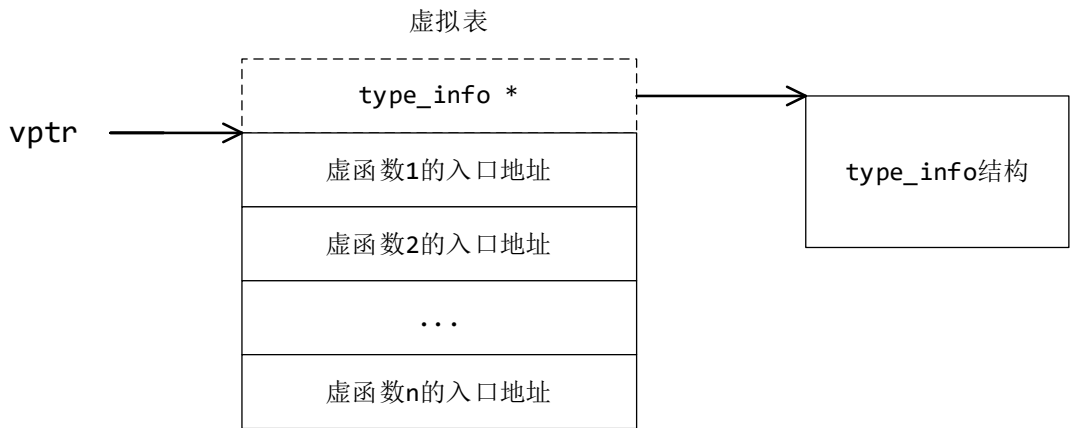


图 18-5 虚函数表中用 `type_info` 结构存储类型信息

正因为如此，使用 `dynamic_cast` 要求类中必须定义至少一个虚函数，以便编译器能够通过虚表获取必要的类型信息。同样地，编译器也可以通过虚表来获取对象的动态类型信息以支持 `typeid` 操作。

18.8.1 `typeid` 运算符

`typeid` 运算符的语法格式为：

- 格式1: `typeid( exp )`
- 格式2: `typeid( type_name )`

运算结果是一个 `const type_info` 类型的对象。`type_info` 是系统内置的结构类型，用于存储类型的相关信息。需要注意的是，`type_info` 的具体结构依赖于具体的编译器实现，因此不同编译器的实现可能会有所不同。对于 `typeid` 运算符：

- (1) 如果操作数是类型的名称，则返回类型名对应的类型信息；
- (2) 如果操作数是指针型表达式，则返回指针型对应的静态类型信息；
- (3) 如果操作数是对象型或引用型表达式，对象中无 `vptr`，则返回静态类型的信息；
- (4) 如果操作数是对象型或引用型表达式，对象中有 `vptr`，则返回动态类型的信息；

例如：

```
1  #include <iostream>
2  #include <typeinfo>
3  using namespace std;
4  class Animal {
5  public:
6      virtual ~Animal( ) = default;
7  };
```

```

8  class Dog :public Animal {
9  };
10 class Cat : public Animal {
11 };
12 int main( ) {
13     int n = 10;
14     int * pn = &n;
15     const int * pcn = &n;
16     Animal * pAni = new Dog;
17     cout << typeid(int).name( ) << endl;    //int
18     cout << typeid(n).name( ) << endl;      //int
19     cout << typeid(pn).name( ) << endl;    //int *
20     cout << typeid(pcn).name( ) << endl;    //int const *
21     cout << typeid(*pcn).name( ) << endl;  //int
22     cout << typeid(Animal).name( ) << endl; //class Animal
23     cout << typeid(Dog).name( ) << endl;   //class Dog
24     cout << typeid(Animal *).name( ) << endl; //class Animal *
25     cout << typeid(Dog *).name( ) << endl; //class Dog *
26
27     cout << typeid(pAni).name( ) << endl;  //class Animal *
28     //若去掉 Animal 中的虚析构, 那么下面的语句将输出 class Animal
29     cout << typeid(*pAni).name( ) << endl; //class Dog
30
31     delete pAni;
32 }

```

18.8.2 dynamic_cast 运算符

dynamic_cast 运算符用于公有继承下的向下类型转换, 目的是将 expression 表示的父类指针或引用安全地转换为子类的指针或引用类型 T, 其语法格式为:

格式: *dynamic_cast*< T > (expression)

运算结果是一个类型为 T 的指针或引用。特别地, 目标类型 T 仅能是指针、引用或 void * 三种形式之一。

- (1) 如果 T 是指针或 void *, 转换成功时返回非空指针; 失败则返回 nullptr。
- (2) 如果 T 是引用类型, 转换成功时返回有效引用; 失败则抛出 bad_cast 异常。

例如:

```

1  #include <iostream>
2  using namespace std;
3  class Animal {

```

```

4  public:
5      virtual ~Animal( ) = default;
6  };
7  class Dog :public Animal {
8  };
9  class Cat : public Animal {
10 };
11 int main( ) {
12     Animal * p1 = new Dog;
13     Animal * p2 = new Cat;
14     Dog * p = dynamic_cast<Dog *>(p1);
15     if (p) {
16         cout << "p1 指向的是一个 Dog." << endl;
17     }
18     else {
19         cout << "p1 指向的不是一个 Dog, 可能是一个 Cat." << endl;
20     }
21
22     try {
23         Dog & dog = dynamic_cast<Dog &> (*p2);
24         cout << "p2 指向的是一个 Dog." << endl;
25     } catch (...) {
26         cout << "p2 指向的不是一个 Dog, 可能是一个 Cat." << endl;
27     }
28     delete p2;
29     delete p1;
30 }

```

运行结果:

```

p1 指向的是一个 Dog.
p2 指向的不是一个 Dog, 可能是一个 Cat.

```

再次强调, 要使用 `dynamic_cast` 运算符, 操作数对应的类中必须至少包含一个虚函数, 以确保存在虚函数表指针。

对于指针类型, 可以通过检查 `dynamic_cast` 的返回值是否为 `nullptr` 来判断转换是否成功。而对于引用类型, 由于引用必须绑定到有效对象, 若转换失败则会抛出 `std::bad_cast` 异常来通知调用者。正因为如此, 在实际开发中, 出于安全性和便捷性的考虑, 使用指针类型的 `dynamic_cast` 更为常见。

18.9 最佳实践

在工程实践中，虚函数被广泛应用。为了正确使用虚函数，以下给出一些建议：

1. 恰当地选用虚或非虚函数。如果父类中的某个实例方法需要由子类提供不同的实现，则应将其定义为虚函数；若该方法的实现对所有子类都是通用的，则应定义为非虚函数。也就是说，在公有继承的情况下，子类会将父类的虚函数作为接口来继承，并可根据需要重写；而对于非虚函数，子类不仅继承了接口，还同时继承了具体实现。例如：

```
1 class Shape {
2 public:
3     virtual ~Shape( ) = default;          //显式定义虚析构函数
4     void show( ) const { std::cout << "面积是" << area( ) << std::endl; }
5     virtual double area( ) const = 0; //纯虚函数
6 };
```

这里的 `show()` 不应被定义为虚函数，因为其具体实现是所有子类共用的。在继承时，子类应当完整地继承 `show()` 的接口及其实现，而不仅仅是接口本身。尽管将 `show()` 定义为虚函数不会影响程序的执行结果，但这并不是规范的设计风格。

2. 使用 `override` 关键字。当子类重写父类的虚函数时，**建议使用 `override` 关键字**。这不仅可以明确表明该函数意图重写父类中的虚函数，还能帮助编译器检测并防止因拼写错误或其他意外情况导致的错误覆盖。例如：

```
1 class Parent {
2 public:
3     virtual ~Parent( ) { }
4     virtual int func( ) const { }
5 };
6 class Child : public Parent {
7 public:
8     virtual int func( int ) { }          //语法能通过，但隐藏了 Parent::func( )
9     virtual int func( int ) const { }    //语法能通过，但隐藏了 Parent::func( )
10    //virtual int func( ) override { } //编译错误
11    //virtual int func( int ) override { } //编译错误
12    virtual int func( ) const override { } //正确
13 };
```

3. 不要重定义父类的非虚函数。根据前述内容，父类中的非虚函数意味着子类在继承时应同时继承其接口和实现。如果在子类中重新定义这些函数，就会违背非虚函数的初衷。若确

实有需求改变行为，更合适的做法是将父类的非虚函数改为虚函数。例如，下面子类 Rectangle 的定义就不符合规范。

```

1  class Polygon {
2  public:
3      Polygon(int sides = 3):mSides(sides) { }
4      virtual ~Polygon( ) = default;
5      void show( ) const { cout << "面积是" << area( ) << endl; }
6      virtual double area( ) const = 0; //纯虚函数
7  protected:
8      int mSides;
9  };
10 class Rectangle : public Shape {
11 public:
12     Rectangle( double w, double h ) : Polygon( 4 ), mWidth( w ), mHeight( h ) { }
13     void show( ) const { cout << "矩形的面积 = " << area( ) << endl; } //重定义，不建议
14     virtual double area( ) const override { return mWidth * mHeight; }
15 private:
16     double mWidth;
17     double mHeight;
18 };

```

4. **不要定义私有虚函数**。在 C++ 中，私有虚函数的处理机制与其他面向对象语言可能存在差异。鉴于团队成员的知识背景各异，禁止使用私有虚函数有助于提升代码的可理解性和可维护性，同时也便于项目的迁移和维护。如果确实需要定义虚函数但又不希望其为公有，可以将其定义为保护成员。
5. 显式定义虚的析构函数。在顶层基类中，如果包含虚函数，则应将析构函数定义为虚函数，这样可以确保对象在销毁时能够正确地执行析构过程，从而避免资源释放不完全的问题。

例如：

```

1  class Parent {
2  public:
3      virtual ~Parent( ) { } //显式定义成虚的
4  };
5  class Child :public Parent {
6  };
7  int main( ) {
8      Parent * p = new Child;
9      delete p;
10 }

```

执行 `delete p;` 时, 会调用析构函数, 即等效于调用 `p->~Parent();`。如果指针 `p` 的静态类型是 `Parent*` 并且 `Parent` 的析构函数不是虚函数, 则会采用静态绑定, 仅调用 `Parent::~~Parent();`, 这将导致派生类 `Child` 中的额外数据成员无法正确释放。相反, 如果 `Parent` 的析构函数被声明为虚函数, 则会使用动态绑定, 在运行时根据 `p` 实际指向的对象类型查找并调用相应的析构函数, 例如 `p->Child::~~Child();`。在 `Child` 的析构函数执行完毕后, 系统会自动调用基类的析构函数 `Parent::~~Parent();`, 从而确保所有资源都能被完整释放。

6. 不要重载虚函数。若类中有重载的虚函数, 表明子类中可以重写(override)这些虚函数, 即使使用 `override` 关键字, 子类中很可能只重写了部分虚函数, 这会隐藏掉父类的其它同名虚函数, 这时子类将不再具有父类的所有接口, 子类将不再"是一种"父类了。除非重载函数带有 `final` 修饰或者类本身就有 `final` 类, 否则不建议重载虚函数。
7. 不要定义虚的赋值函数。赋值函数的本意是进行同类型对象间的赋值操作, 一旦定义虚的赋值函数, 那么赋值行为可能变得荒唐。例如:

```

1  #include <iostream>
2  class Animal {
3  public:
4      virtual ~Animal( ) { }
5      virtual Animal & operator=( const Animal & ani ) {
6          std::cout << "Animal = Animal" << std::endl;
7          return *this;
8      }
9  };
10 class Cat : public Animal {
11 public:
12     virtual Cat & operator=( const Animal & ani ) override {
13         std::cout << "Cat = Cat" << std::endl;
14         return *this;
15     }
16 };
17 class Dog :public Animal { };
18 int main( ) {
19     Cat cat;
20     Dog dog;
21     Animal & ani = cat;
22     ani = dog;    //输出 Cat = Cat
23 }
```

这里将允许将 `dog` 赋值给 `cat`，这明显不合理。

8. 使用抽象类和接口类。抽象类和接口类通过定义通用的行为或约定，隐藏了具体的实现细节，降低系统的复杂性。相对于具体类，它们具有更高的代码抽象层次，能够更好地应对变化并促进代码复用，从而增强整个系统的灵活性和可维护性。